Reverse-Harness: Design Patterns for Runtime, Agent-Initiated Capability Provisioning under Governance

Rameswaran Mohan
Independent Researcher
India
rameswaranmohan1993@gmail.com

Abstract

Supervised-agent runtimes provision an agent's harness—its tools, skills, permissions, compute, and helpers—at design time, before the task is known. The system must predict what the agent will need. We argue this push model is mismatched to how agents actually get stuck: an agent's real needs are discoverable only at runtime, so a frozen harness either over-provisions (costly and over-broad) or under-provisions (the agent stalls with no recourse). We build and evaluate *Reverse-Harness*, a design that inverts provisioning from design-time push to runtime pull: the executing agent requests the capabilities it lacks, and an always-on Governor materializes them *proportionately*—resolving the reversible majority for free and reserving human escalation for the dangerous-and-irreversible. We make five contributions. (C1) We characterize the push/pull mismatch and the inversion that resolves it under a single-owner, local-deputy threat model. (C2) We abstract the design as a pattern language of six patterns that make self-provisioning bounded and auditable and that transfer to other agent runtimes. (C3) We identify and instrument *pull-decision quality*—whether the agent knows it is blocked and asks for the right capability—as the load-bearing, previously unexamined hinge of self-provisioning, and the paper's central empirical target. (C4) We show that pull-provisioning spans six capability families (tool, skill, access, compute, sub-agent, MCP) under one governance path—covering *both* ways an agent acquires a capability at runtime, *synthesis* (forging new code, tool) and *attachment* (connecting a vetted external MCP server)—where most prior tool-creation work covers tools alone; we characterize each family by what the reference implementation actually materializes, which differs across families—tool, MCP, and compute end-to-end, sub-agent as real parallel execution (opt-in), skill indirectly, and access as arbitrated-and-logged—so the count of six is always paired with this materialization gradient rather than asserted flatly. (C5) We design a controlled Capability-Gap Benchmark, judged by an independent oracle, to measure pull-decision quality, efficacy, governance cost, and the narrow safety invariants the design enforces (HIGH-risk requests always escalate; the judge can only downgrade; failures fail safe)—invariants, not a containment guarantee the single-owner model does not claim. We instantiate the patterns in systemu, an open-source single-owner automation runtime (evaluated at release v0.9.41), and report a 179-trial benchmark under this protocol.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Multi-agent systems*; • **Security and privacy** → *Systems security*.

Keywords

LLM agents, capability provisioning, design patterns, agent runtimes, governance, least privilege, auditability, self-provisioning

1 Introduction

A supervised-agent runtime—the software that drives a large-language-model (LLM) agent through a task under a supervising authority—must give the agent a *harness*: the tools it can call, the skills (procedural knowledge) it can consult, the permissions and resource access it holds, the compute budget it may spend, and the helper sub-agents it may enlist. In the prevailing design, the runtime assembles this harness at design time, before the task is known. The system, or its operator, must *predict* what the agent will need.

1.1 The push/pull mismatch

Design-time provisioning is a *push*: the system pushes a fixed set of capabilities at the agent in advance. This is mismatched to how agents actually encounter their needs. An agent's real requirements are discoverable only at runtime—after it has read the task, attempted an approach, and found a gap. Predicting that gap ahead of time forces an unhappy choice. Provision generously, and the harness over-provisions: the agent holds powers it never uses, widening the attack surface, raising cost, and violating least privilege. Provision conservatively, and the harness under-provisions: when the agent hits a missing capability mid-task, it has no recourse. It cannot ask for what it lacks; the runtime can only let it fail, stall, or hallucinate a workaround.

This is not a tuning problem to be solved with a better predictor. The information that resolves it—what this particular task needs—does not exist until the agent is already running. The mismatch is structural: the deciding party (the system, at design time) and the knowing party (the agent, at runtime) are separated in time.

The natural resolution is to invert the flow. Instead of the system pushing capabilities it guesses the agent will need, let the executing agent *pull* the capabilities it discovers it lacks, and let the system materialize them under governance. We call this design *Reverse-Harness*. Inversion alone is not safe—an agent that can grant itself arbitrary powers is worse than one that is merely under-provisioned—so the contribution is not the inversion in isolation but the inversion *plus* the governance discipline that keeps it safe, bounded, and auditable.

1.2 The threat model: a single-owner local deputy

The governance discipline that self-provisioning needs depends entirely on who the agent is acting for. We are explicit about this

up front, because it determines which guarantees are appropriate to claim and which would be overclaims.

Our setting is a *single-owner, local automation tool*. The agent runs on the owner's own machine, as the owner's deputy, with the owner's own permissions. There is one principal: the human owner. The agent is not a mutually distrustful tenant in a shared system, and it is not facing an adversary who controls part of the runtime. It is doing work the owner could do by hand, faster.

This threat model is deliberately *not* multi-tenant zero-trust. Under single ownership, a containment story—forced lease expiry, a default-deny sandbox that jails the agent from the filesystem and network—would both overclaim and misfire. It overclaims because, when the agent legitimately holds the owner's permissions, there is no jail to escape from; and it misfires because the friction it imposes fights the very automation the tool exists to provide. The honest and, we argue, more useful question is not "how do we contain an untrusted agent" but *how little governance does self-provisioning actually need, and where does friction stop being worth it?* Our answer: *proportionate governance*—frictionless for the reversible majority, and a deliberate, light touch only for the genuinely dangerous-and-irreversible.

The spine of the paper follows from this framing. The two load-bearing concerns are (i) *pull-decision quality*—does the agent correctly recognize that it is blocked and ask for the right thing?—and (ii) *proportionate governance*—does the arbitration resolve the easy majority cheaply while reliably escalating the dangerous minority? Containment is explicitly *not* the spine.

1.3 Contributions

We make five contributions.

- C1 — The inversion. We show that capability provisioning for supervised agents can be inverted from design-time push to runtime pull, governed *proportionately*: every self-request flows through one authority that resolves the reversible majority for free and reserves friction (human escalation) for the dangerous-and-irreversible (§1.1, §3).
- C2 — A pattern language. We abstract the design as a language of six patterns that make the inversion bounded and auditable. The patterns are the reusable core, stated in full pattern form and transferable to other agent runtimes; well-known mechanisms (reference monitors, capabilities, control/data-plane separation) are cited as prior art, with only the novel application claimed (§3).
- C3 — Pull-decision quality (the spine). The paradigm's load-bearing assumption is that the agent *knows when it is blocked and asks for the right thing*. We instrument it and report two complementary measures: the *recognition rate* (a correct-kind request fired per pull run), cross-validated against recovery; and the full *request-outcome taxonomy*—the evaluated v0.9.41 build reconciles every request to a terminal governor-ledger disposition, so we report grant disposition (used/unused/not-applicable-by-kind), premature requests, wasted requests, and over-delegation denied at the per-run cap, re-derived from the durable per-trial ledgers. This is the previously unexamined hinge of self-provisioning and the central empirical contribution (§5).
- C4 — Breadth of self-provisioning. Pull-provisioning spans six capability families—tool, skill, access, compute, sub-agent, and MCP—under one governance path, where most prior tool-creation work covers tools alone. Crucially, the same arbiter governs *both* ways an agent can acquire a capability at runtime: *synthesis* (tool—forging new code) and *attachment* (MCP—connecting a vetted external Model Context Protocol server). The count of six is paired throughout with a *materialization gradient*: we characterize each family by what the reference implementation actually materializes—tool, MCP, and compute end-to-end, sub-agent as real parallel execution (opt-in, behind a default-off flag), skill indirectly, and access as arbitrated-and-logged with enforcement on the Docker backend only. The end-to-end evidence is therefore strongest for tool, MCP, and compute, and the breadth claim is made at the level each family supports (§4).
- C5 — Empirical validation. We design a controlled Capability-Gap Benchmark, judged by an *independent* oracle, to measure pull-decision quality (primary), efficacy (does pull recover gap-blocked tasks versus a push baseline?), governance cost (the proportionate-arbiter overhead, with the measured deterministic-versus-LLM split), and the narrow safety invariants the design enforces—HIGH-risk requests always escalate, the judge can only downgrade, and failures fail safe—which are bounded invariants, not a containment guarantee the single-owner threat model does not claim (§5).

The patterns are validated by **systemu**, an open-source single-owner automation runtime whose reference implementation (v0.9.41) we describe in §4. The remainder of the paper proceeds as follows. §2 situates the work among supervised-agent runtimes and states the design-time-frozen-harness problem with a concrete failure. §3 presents the six patterns. §4 describes the reference implementation and maps patterns to code. §5 gives the evaluation protocol. §6–§8 discuss implications, related work, and future directions.

2 Background and Motivation

2.1 Supervised-agent runtimes

An LLM agent does not act in a vacuum. It runs inside a *runtime* that drives an action loop: at each step the runtime gives the model the task, the relevant context, and an action vocabulary; the model returns a chosen action (reason, call a tool, ask a question, report done); the runtime executes it, observes the result, and loops. In a *supervised* runtime a separate authority oversees this loop—scheduling steps, enforcing budgets, gating risky actions, and recording what happened—so the agent's autonomy is bounded by an external party rather than left to the model alone.

Across these runtimes the agent's *harness* is fixed before the loop begins. The set of callable tools, the loaded skills, the granted permissions and resource access, the iteration and spend budget, and the ability to spawn helpers are all decided in advance—by a configuration file, an operator, or a planning pass—and frozen for the duration of the task. The agent consumes its harness; it does not shape it. This is the design-time *push* of §1.1.

2.2 The design-time-frozen-harness problem

Freezing the harness up front forces the system to commit to a capability set before the information that would justify it exists. The relevant information is *task-local and discovered late*: which exact tool, which permission, which extra increment of budget the agent needs only becomes apparent once the agent is mid-task and blocked. Three consequences follow.

Over-provisioning. To avoid stalls, operators provision broadly: the agent is handed a wide tool belt and generous permissions "just in case." Most of it goes unused on any given task. The cost is paid in the standard currency of least-privilege violations—a larger attack surface, more ways for a confused or prompt-injected agent to do harm, higher token and dollar cost from carrying unused affordances in context, and a weaker audit story (every run looks maximally capable, so nothing stands out).

Under-provisioning. The opposite discipline—provision only what is clearly needed—fails differently. When the agent hits a genuine gap, the frozen harness offers no path forward. The runtime can let the agent fail outright, let it spin (re-attempting the impossible until a loop guard trips), or let it improvise an unsound workaround. None of these is the right outcome, which is simply: *get the missing capability, under appropriate scrutiny, and continue.* A frozen harness structurally cannot offer that.

The predictor does not help. One might hope a better up-front predictor—a planning pass that inspects the task and provisions accordingly— closes the gap. It cannot close it in general, because the deciding moment (provisioning) precedes the knowing moment (the agent discovering, through attempted execution, what it actually lacks). A planner can improve the guess; it cannot make the guess unnecessary. The structural fix is to move the provisioning decision to the moment the need is known—i.e., to invert push into pull.

2.3 A concrete failure

The reference runtime began as a frozen-harness engine (the "legacy" intent engine, still selectable via a configuration flag, which we use as the push baseline in §5). Its behavior on under-provisioned tasks motivates the inversion concretely.

Consider an automation task—in the runtime's domain model, a *scroll* (a stored, repeatable instruction)—that requires a capability the frozen harness does not include: say, a tool to transform a specific document format that no pre-provisioned tool handles. The legacy engine has no way for the agent to obtain the missing tool. It cannot forge one, cannot request one, and cannot escalate the gap to the operator as an actionable request. Its only honest response is to *park* the scroll: mark it blocked and stop. The capability gap is real, the task is otherwise well within reach, and the human is left to diagnose the stall after the fact and re-provision by hand—exactly the information-too-late loop the frozen harness creates.

This parked-scroll failure is the unit case the rest of the paper addresses. Under Reverse-Harness the same situation has a different ending: the agent recognizes it is blocked, issues a runtime request for the missing capability, the governance layer arbitrates the request (auto-granting it if it is low-risk, or escalating it to the operator if it is not), and—if granted—the agent continues with the capability in hand. The benchmark in §5 manufactures this situation deliberately and at scale: each task is solvable *only if* the agent pulls a specific, withheld capability, turning the anecdote into a controlled measurement.

3 The Pattern Language

We present the design as a language of six patterns. Each is stated in full pattern form—*Name*, *Context*, *Problem*, *Forces*, *Solution*, *Consequences*, and *Known Uses* in the systemu reference implementation. The patterns compose in three layers: *inversion* (P1–P2) turns push into pull; *authority and arbitration* (P3–P5) make self-provisioning safe and reliable; and *accountability* (P6, with the attribution half of P4) makes it auditable.

Throughout, well-known security and systems techniques—reference monitors and complete mediation, capability-based security and leases, fail-safe defaults, and control-plane/data-plane separation—are cited as *prior art*. We do not claim them. What we claim is their specific application to runtime, agent-initiated provisioning, together with a small number of novel kernels called out explicitly under each pattern.

Prior art vs. contribution. To pre-empt the reading that these patterns merely relabel known ideas, Table 1 accounts, per pattern, for the lineage each builds on and what (if anything) is genuinely new here. We state plainly where a pattern is mostly a *recombination*: P5 and P6 contribute little primitive novelty and locate their value in the composition and context, not the mechanism. The defensible novelty concentrates in the *inversion* of provisioning direction as a first-class runtime verb (P1–P2), in instrumenting pull-decision quality as the load-bearing hinge (P2), and in two arbitration kernels—self-request asymmetry and downgrade-only judgment (P3)—applied uniformly across the six capability families. Each row's claim is restated and defended in the corresponding subsection below.

3.1 P1 — Pull-Provisioning (root)

Context. A supervised agent runtime provisions a harness—tools, skills, permissions, compute, helpers—for a task.

Problem. The agent's real needs are discoverable only at runtime. A design-time-frozen harness therefore either over-provisions (insecure and costly) or under-provisions (the agent stalls with no recourse), as established in §2.2.

Forces. Predictability and auditability pull toward a fixed, known-in-advance harness; adaptability pulls toward a harness that changes with the task. Least privilege pulls toward provisioning little; autonomy pulls toward provisioning enough to finish. The system *guessing* up front is in tension with the agent *knowing* mid-task.

Solution. Invert provisioning. The executing agent requests capabilities at runtime, at the moment it discovers a gap; the system materializes them under governance. Provisioning becomes demand-driven and least-privilege by default: the agent starts with little and acquires exactly what the task proves it needs, when it proves it needs it.

Consequences. Positive: the harness is adaptive and least-privilege by default; over-provisioning disappears because nothing is granted

Table 1: Per-pattern prior-art accounting. "Builds on" lists the lineage we *cite as prior art and do not claim*; "Novel here" states what is new, or marks the pattern as mostly a recombination whose value is in context. We deliberately attach no citation to LLM-as-judge: no suitable key is in our bibliography, so we describe it in prose only.

Pattern	Builds on (prior art, cited; not claimed)	Novel here (or: recombination)
P1	Runtime agent tool creation [9, 15]; least privilege and fail-safe defaults [11].	*Inverting* provisioning direction (push→pull) as a first-class runtime concern, generalized beyond tool creation to six capability families (spanning both synthesis and attachment) under one governance path.
P2	Reasoning-and-acting agent loops [13, 16]; the command / request-object discipline (prose, no key).	`REQUEST_HARNESS` as the *dual of the tool-call verb*, with inline-grant-or-suspend semantics; and instrumenting *pull-decision quality* as the load-bearing, measured hinge.
P3	Reference monitor / complete mediation and fail-safe defaults [11]; capability discipline [8]; risk-tiered human-in-the-loop gating [1, 14].	Two kernels: *self-request asymmetry* (a self-selected capability is scored as more dangerous) and *downgrade-only* judgment (the judge may only move a verdict toward the safe default); a default-deny, three-band loop applied *uniformly* across all six families.
P4	Capability / object-capability security [8, 11]; leases as a grant primitive [4].	Mostly a *recombination*: attribution + one-click revocation in place of expiry, deliberately scoped to the single-owner threat model. We provide provenance and revocation, *not* unforgeable tokens or auto-expiry; novelty is the scoping choice, not the primitive.
P5	Control-plane / data-plane separation [7]; human-in-the-loop oversight [1, 14]; LLM-as-judge (prose, no key).	Mostly a *recombination*: applying the control/data split to an LLM-in-the-loop runtime where the slow/flaky component is a language model and the fail-safe direction is escalation. The *composition*, not the principle, is the contribution.
P6	Append-only / event-sourced audit logs [3]; visibility and accountability for AI agents [2].	Mostly a *recombination*: an event-sourced ledger scoped to reconstructing a *self-assembled* harness, carrying the deterministic-vs-LLM decision provenance. The target (self-provisioning) is new; the logging primitive is not.

speculatively; the audit trail records intent (what the agent asked for and why), not just capability. *Negative:* inversion requires a runtime authority, an arbitration mechanism, and introduces new failure modes—request thrash, abuse, and the question of whether the agent asks well at all. These negatives are exactly what P2–P6 address.

Known uses. The systemu reverse-harness (origin v0.9.7; reference implementation v0.9.41, §4). P1 is the root; the remaining patterns exist to discharge its negative consequences.

3.2 P2 — Inverse-of-Invocation Verb (`REQUEST_HARNESS`)

Context. The agent loop has an action vocabulary (reason, call a tool, report, ask). Pull-provisioning (P1) needs a way to be *expressed* within that vocabulary.

Problem. How does the agent *ask* for a capability such that the request is auditable, suspendable, and forced through a single chokepoint?

Forces. Three designs compete. (i) A *meta-tool*—"call the provision-me tool"—rides existing tool-call machinery but is semantically muddy (a tool that creates tools) and must return synchronously, which fights suspend/resume. (ii) A *supervisor-mediated need-signal*—the agent hints, an external observer infers the need—reintroduces the very "observer guesses" gap that P1 closes. (iii) A *first-class verb* is cleaner but changes the agent's decision schema.

Solution. Add `REQUEST_HARNESS` as a first-class loop primitive: the *dual* of the tool-call verb. Where a tool call says "use a capability I have," `REQUEST_HARNESS` says "provision a capability I lack." It carries a structured request—`kind`, `spec`, `rationale`, `fallback`, and a `blocking` flag. The non-obvious design rule is the *blocking semantics*: the runtime either grants inline (the agent continues this step with the capability in hand) or *suspends* the execution until the request is resolved. It never lets the agent continue and splices a late grant into a loop that has already moved on. Inline-grant-or-suspend keeps the agent's view of its own capabilities consistent at every step.

Consequences. Positive: a single auditable chokepoint for all self-provisioning, and a natural suspend/resume boundary. *Negative:* the agent's decision schema and system prompt must change to teach *when to pull*—as distinct from when to keep trying, when to think, and when to fail. Pull-decision quality (C3) is precisely the measure of how well the agent learns that distinction.

Known uses. `REQUEST_HARNESS` and `ASK_OPERATOR` as loop verbs in the shadow runtime; the `HarnessRequest` request object in the core model layer, carrying `kind`, `spec`, `rationale`, `fallback`, and `blocking`. *(Cites: the command and request-object patterns as prior art.)*

3.3 P3 — Risk-Tiered Self-Provisioning Arbitration

Context. Requests of widely varying danger arrive at runtime, from "reuse a tool I already have" to "forge new executable code" to "write to a secret."

Problem. Human-gating every request stalls autonomy; auto-granting every request is unsafe; static allow/deny rules cannot reason about novel requests they were never written for.

Forces. Friction, safety, and autonomy are mutually in tension. Determinism (fast, predictable, cheap) is in tension with flexibility (handles the novel case). Judgment has a cost—latency, money, and the risk of being wrong.

Solution. Classify each request by a risk function $risk = f(kind, reversibility, cost, scope)$ into three bands. **LOW** requests are auto-granted by a deterministic rule (no model in the loop). **MEDIUM** requests go to an LLM judge, which grants only if it is confident *and* policy allows, and escalates otherwise. **HIGH** requests always escalate to the operator. The default throughout is deny: anything not affirmatively granted is withheld. Two kernels are novel and are what we claim. *(a) Self-request asymmetry:* a capability the agent *requested* is treated as *more* dangerous than the same capability pre-provisioned, because the agent chose it—the act of self-selection is itself a risk signal. *(b) Downgrade-only judgment:* the LLM judge can only ever move a verdict *toward* the safe default (deny or escalate). It can never grant beyond policy or "open a hole." A confident judge can confirm a grant the policy already permits; a hesitant or failing judge can only fall back to escalation.

Consequences. Positive: safe autonomy with bounded human involvement—the operator sees only the genuinely dangerous minority. *Negative:* the design requires a policy, a judgment layer (P5), and escalation machinery; and the risk scoring must stay conservative, since the whole safety argument rests on HIGH-risk requests never slipping into the auto-grant band. *(Cites: the reference-monitor / complete-mediation principle and Saltzer–Schroeder fail-safe defaults as the generic foundation; only kernels (a) and (b) are claimed as novel.)*

Known uses. The deterministic `harness_arbiter` (pure risk-scoring and the LOW/MEDIUM/HIGH band table); the `harness_judge` (the LLM judge, which grants only at self-reported confidence ≥ 0.6 and otherwise escalates—an instance of downgrade-only); the `harness_policy` object; and the Governor that composes them (§4).

3.4 P4 — Attributed, Revocable Self-Grants

Context. A single-owner deputy forges capabilities for itself mid-run—most commonly tools. Those capabilities are useful again on the next task.

Problem. The problem is *not* "self-granted powers must expire." Under the single-owner threat model (§1.2) forced expiry destroys the persistent automation the product exists to provide: a tool that worked yesterday should still work today. The real problem is *accountability*. The owner must be able to see exactly what the agent gave itself, attribute each capability to the run that created it, and revoke any of it in a single action.

Forces. Automation and persistence (keep what works) pull against oversight (see and undo). For a single owner the right axis is *visibility plus easy undo*, not ephemerality. Ephemerality would impose friction (re-granting the same useful tool every run) to buy a containment property that single ownership does not need.

Solution. Self-built capabilities *persist*: a forged tool is deployed and enabled like any other tool, and remains available across runs. What makes persistence safe under oversight is *provenance plus one-click revocation*. Each self-built capability records who built it (it was forged by the agent, not the operator) and *which run* built it (the originating execution id), and the harness ledger (P6) records its lease lifecycle. The owner gets an "agent-built" view that filters the registry to exactly the agent-forged capabilities, and the existing per-tool disable toggle *is* the revoke action. Trust compounds by simple use—a tool that keeps working keeps existing—and the owner can disable anything, at any time, instantly.

Consequences. Positive: zero added friction; the agent keeps what it earns; full attribution and instant revocation. *Negative:* there is no automatic containment—a self-built tool persists until the owner disables it. We argue this is acceptable precisely under the single-owner model: the capability is code-reviewable before it is enabled, attributable to its originating run, and owner-revocable in one click. We explicitly *drop* the earlier and stronger claim that self-grants "auto-revoke" or "never leak across runs"; that claim was both false as implemented and hostile to the tool's purpose.

Known uses. The forged-by-agent provenance flag on the tool model, plus the originating-execution provenance field set at the runtime forge site; lease-mint and lease-revoke events in the harness ledger; and the Tools Registry disable toggle together with the "agent-built" filter and badge. *(Cites: capability-based security and object capabilities as related framing, not as the claimed mechanism—we provide attribution and revocation, not unforgeable capability tokens.)*

3.5 P5 — Off-Path Judgment

Context. The runtime has a reliability-critical dispatcher (it must schedule loop steps deterministically and never wedge) and, at the same time, a need for LLM-based judgment (P3's MEDIUM band).

Problem. An LLM call is slow, occasionally flaky, and can blow a budget. If such a call sits on the scheduling-critical path, a single bad model call can jeopardize the whole system—stalling not just one request but the dispatcher that drives every execution.

Forces. Intelligence (model judgment) is in tension with reliability (deterministic scheduling). The desire for always-on judgment is in tension with the requirement that dispatch never block.

Solution. Split the layers. Keep the mechanical dispatcher LLM-free and unbreakable: it does scheduling, budgeting, and dispatch with no model in the loop, so it cannot be made slow or flaky by one. Isolate all model judgment in a separate, always-on authority. The two communicate through a narrow interface, and—critically—judgment failures *fail safe*: a judge that times out, errors, or returns

garbage yields an escalation, never a block of the dispatcher and never a default grant. Scheduling is thus never hostage to a model, while judgment can be always-on at acceptable cost.

Consequences. Positive: scheduling is never held hostage by an LLM; judgment can be always-on affordably; the safety argument is simpler because the failure mode of the intelligent layer is contained to "escalate." *Negative:* two layers must be maintained with a clear contract between them. *(Cites: control-plane/data-plane separation as the generic principle; the novelty is its application to an LLM-in-the-loop agent runtime, where the "slow, flaky" component is a language model and the fail-safe direction is escalation to a human.)*

Known uses. The LLM-free Supervisor (the mechanical dispatcher) versus the Governor (the always-on judgment authority, which evolves the prior execution-mind component); the resume-after-grant re-enqueue path by which a suspended execution rejoins the dispatcher once its request is resolved (§4).

3.6 P6 — Provenance Ledger

Context. Over a run, an agent assembles its own harness through many grants—tools forged, budget extended, helpers spawned, access arbitrated.

Problem. After the fact: *what* capabilities did the agent give itself, and *who or what* approved each one? Self-provisioning without a record of this is fundamentally unauditable—the operator cannot reconstruct what the autonomous system did to itself.

Forces. Autonomy is in tension with accountability; the more freely the agent provisions, the more there is to account for. Completeness is in tension with storage cost, though for this workload the records are small relative to the LLM traffic.

Solution. An append-only, per-execution ledger that records every mutation of the self-assembled harness: each request, the verdict it received (and whether that verdict was decided deterministically or by the LLM judge), each lease minted and revoked, each materialization outcome, and the provenance of each artifact. The property we aim for is reconstructability: from the ledger alone, the agent's entire self-assembled harness for a run can be rebuilt. This is the accountability of autonomy, not merely "a log"—the ledger is the artifact that lets a human review, after the fact, exactly what the agent decided to give itself and on whose authority.

Consequences. Positive: complete auditability; direct support for incident review and for the operator trust that makes self-provisioning acceptable at all. *Negative:* the ledger must capture *every* mutation—completeness is the property, and a single un-logged grant breaks the reconstructability guarantee, so the writing discipline must be exhaustive (and is what the evaluation validates for P6).

Known uses. The append-only per-execution harness ledger (one JSONL file per execution under the vault) and the harness-review dashboard surface that renders it for the operator. The ledger carries the structured deterministic-versus-LLM decision field that also powers the governance-cost measurement in §5. *(Cites: append-only and event-sourced audit logs as prior art.)*

4 Reference Implementation

We validate the pattern language with **systemu**, an open-source single-owner automation runtime. This section describes the reference implementation (v0.9.41), characterizes the six capability families *honestly* by what the code actually materializes, and maps each pattern to the components that realize it. The intent throughout is to state what holds and to mark what does not, so that the claims in §3 and the measurements in §5 rest on the system as built rather than as aspired to.

4.1 Runtime shape

The execution loop runs inside a *shadow runtime*: per loop step the model chooses an action from a vocabulary that includes reasoning, tool calls, the two pull verbs (`REQUEST_HARNESS` and `ASK_OPERATOR`), and a terminal report. Two authorities oversee the loop, realizing Off-Path Judgment (P5):

- The **Supervisor** is the mechanical dispatcher. It schedules steps, enforces budgets, and reconciles terminal state with *no LLM in its critical path*, so a slow or flaky model call cannot wedge it.
- The **Governor** is the always-on pull authority. It is constructed per execution and is the single point through which every `HarnessRequest` flows. It arbitrates requests, materializes grants, and writes the ledger.

4.2 The pull path: arbiter, judge, Governor

When the agent emits a `REQUEST_HARNESS`, the resulting `HarnessRequest`—carrying `kind`, `spec`, `rationale`, `fallback`, and the `blocking` flag—reaches the Governor, which arbitrates it in two stages (Risk-Tiered Arbitration, P3).

Deterministic arbiter. A pure, side-effect-free `harness_arbiter` scores the request into a LOW/MEDIUM/HIGH risk band from a fixed table over (kind, sub-case). Representative rows: reusing an already-enabled tool is LOW and auto-granted; forging *new* tool code is HIGH and always escalates; a budget increase within the ceiling is LOW while one over the ceiling is HIGH; reading a whitelisted resource is LOW while a non-whitelisted read is MEDIUM and a write/secret/network access is HIGH; spawning a sub-agent within depth and budget is MEDIUM, and beyond them HIGH. The arbiter never grants beyond policy and flags genuinely ambiguous MEDIUM cases for the judge by setting a "needs LLM judgment" signal.

LLM judge. For flagged MEDIUM requests the Governor calls the `harness_judge`. The judge is *downgrade-only*, the P3 kernel: it returns GRANT only when it both parses a clear decision *and* reports confidence at least 0.6; a missing client, malformed output, an error, or confidence below the threshold all yield ESCALATE with no lease. The judge can confirm a grant the policy already permits, but it can never open a hole. This is the mechanical realization of "the intelligent layer can only ever move toward the safe default."

Verdicts. The Governor returns a `HarnessVerdict` carrying the decision (GRANT/DENY/ESCALATE), the risk band, any DENY alternatives handed back to the agent so it can adapt, and a structured `decided_by` field recording whether the verdict was resolved

Table 2: The six capability families as materialized in systemu v0.9.41. "Materialization" is what the granted request actually produces in the run.

Family	Status	What a grant produces
Tool	end-to-end (synthesis)	forged tool, deployed and callable this run
Mcp	end-to-end (attachment)	external server connected; its tools callable this run
Compute	end-to-end (budget)	added iteration / think budget, live
Skill	indirect	skill persisted; agent then loads it
Subagent	real, parallel (opt-in)	child runtimes spawned concurrently
Access	arbitrated, logged	verdict + ledger; enforcement Docker-only

`deterministic`ally or by the `llm` judge. This field is what makes the deterministic-versus-LLM cost split in §5 a direct measurement rather than an estimate.

4.3 The suspend/resume rail

The `blocking` semantics of P2 are realized by a suspend/resume rail. On a blocking ESCALATE, the runtime snapshots and parks the execution rather than letting it proceed without the capability. When the request is resolved—the operator approves, or a grant is otherwise materialized—the daemon reconciler re-enqueues the parked execution through a resume-after-grant path; the grant is replayed into the loop, and the agent continues from where it paused. The rail embodies the "inline-grant or suspend; never continue-and-splice" rule: a late grant is applied only through a controlled resume, never injected into a loop that has already moved on.

4.4 The six capability families, characterized honestly

A central honesty commitment of this paper is to state, per family, exactly what the reference implementation materializes. Table 2 is the honest characterization; the prose below expands each row. The two end-to-end synthesis- and attachment families—tool (forge new code) and mcp (connect a vetted external server)—are the two halves of the acquisition layer, arbitrated by the same Governor.

Tool — end-to-end (synthesis). A granted tool request runs the forge spine: the proposed tool is forged, deployed (status deployed and enabled), and becomes callable *within the same run*. This is the fully materialized, load-bearing *synthesis* family and the one most exercised by self-provisioning.

Mcp — end-to-end (attachment). The counterpart to synthesis: rather than forging code, a granted MCP request *attaches* a vetted external capability. The request carries a server connect-spec (stdio command or remote URL); the arbiter tiers it—re-attaching a previously-approved or allow-listed server is a LOW-risk grant, an unrecognized external server escalates HIGH, and a connect-spec pointing at a private or loopback address is denied outright as an SSRF attempt. On a grant the runtime connects, discovers the server's tools, pins each tool's definition hash (a drift on a later call fails closed), and registers them as namespaced, callable tools *within the same run*; lease revocation un-registers them. This is the second fully-materialized family and, with tool, completes the acquisition layer—one Governor over both *making* and *borrowing* a capability.

Compute — end-to-end (budget). A granted compute request increases the execution's iteration/think budget in place; the agent immediately has more budget to spend. Fully materialized. Two prerequisites for this to be *usable* were fixed during evaluation (§5.3): the per-iteration payload now surfaces the remaining budget (so the agent can perceive the gap and request), and an under-specified grant now defaults to a non-zero budget rather than zero (so the request is not a no-op).

Skill — indirect. A granted skill request *persists* a skill candidate and surfaces it to the agent; the agent must then explicitly load the resource to use it. The capability is provisioned, but consumption is a second, agent-driven step rather than an automatic injection—hence "indirect."

Subagent — real and parallel. A granted sub-agent request spawns a *fleet* of child runtimes. Each child is a full shadow runtime over its own shadow and activity, built to inherit the parent's tools and skills *minus* the delegation tools—a hard non-recursion barrier at the next level, with depth additionally capped at one for this build. Children run concurrently under an `asyncio` semaphore sized from the configured maximum-concurrent-children cap (and a hard ceiling on fan-out width), each wrapped in a per-child timeout, gathered so that one child's failure or timeout never kills its siblings. Results are then *collated* under one operator decision: *partial failure is not total failure.* The collation reports what each successful child produced and explicitly names what failed and is therefore missing, and it sums per-child cost signals so the parent retains cost visibility across the fan-out. This family was previously a one-shot LLM call; it is now real parallel execution, which is what makes the breadth claim (C4) honest for sub-agents. Because it is the only behavior-changing capability, it is gated behind a default-off flag (`SYSTEMU_DELEGATE_USE_PARALLEL`): enabled, a granted sub-agent request spawns the fleet; disabled—the default—the legacy one-shot path is used. The parallel materialization is therefore *opt-in* in this build, and the evaluation reports it as such.

Access — arbitrated and logged. A granted access request is arbitrated and recorded in the ledger, but its *enforcement substrate* exists only on the Docker backend (a read-only mount, no vault). The default local backend runs tools via subprocess with environment scrubbing and *no filesystem or network jail*—by design, because under the single-owner threat model the agent acts with the owner's own permissions (§1.2). We therefore state access as *arbitrated + logged*: the request, the verdict, and the decision are first-class and audited, while default-deny sandbox enforcement is an optional Docker hardening noted as future work, not a shipped guarantee. The evaluation measures access tasks at the level of *arbitration* (request→verdict), not materialization.

Table 3: Pattern-to-code map for the systemu v0.9.41 reference implementation.

Pattern	Realizing components
P1 Pull-Provisioning	the reverse-harness pull path (whole pipeline)
P2 Inverse Verb	`REQUEST_HARNESS` / `ASK_OPERATOR`; `HarnessRequest`; suspend/resume rail
P3 Risk Arbitration	`harness_arbiter` (deterministic); `harness_judge` (downgrade-only); `harness_policy`; Governor
P4 Attributed Grants	forged-by-agent + execution-id provenance; ledger lease events; "agent-built" filter
P5 Off-Path Judgment	LLM-free Supervisor vs. Governor; resume-after-grant re-enqueue
P6 Provenance Ledger	per-execution harness ledger; decision audit; harness-review dashboard

4.5 Accountability: the harness ledger and provenance

The Provenance Ledger (P6) is an append-only JSONL file per execution under the vault. Each line records a request, its verdict (including the structured `decided_by` field), the materialization outcome, lease-mint and lease-revoke events, and artifact provenance. Self-built tools carry a forged-by-agent flag and, when forged at runtime, the originating execution id; the design-time forge sites legitimately carry no execution id, which is the correct distinction between runtime-self-forged and operator-forged. The Tools Registry exposes an "agent-built" filter and badge, and the existing per-tool disable toggle is the revoke action—together realizing the attribution-plus-revocation half of P4.

A separate per-iteration *decision audit* (one JSONL row per loop step) records the chosen action, the model's reasoning, and the live loop-health signals that were active at decision time—consecutive reasoning steps, loop-guard state, stuck-round count, and the research-read and tool-failure streaks—plus, on a `REQUEST_HARNESS` step, the request's kind, self-reported confidence, and the number of tool attempts that preceded it. These blockage signals are loop-local and are not otherwise persisted; capturing them is what makes it possible to answer, offline, whether a given request *followed genuine blockage*—the basis of the pull-decision-quality measurement in §5. The decision audit is pure observability: it changes no loop logic and is fully backward-compatible.

4.6 Pattern-to-code map

Table 3 maps each pattern to the components that realize it.

4.7 Invariants

The implementation preserves a small set of invariants that the safety and accountability arguments depend on: the Supervisor stays LLM-free; only verified tool-call results may credit a task's objectives (instrumentation never credits); default-deny arbitration is unchanged by any of the observability additions; each execution—including each spawned child—gets its *own* Governor with no shared grant state; the ledger is append-only and new event types are purely additive; and all instrumentation is backward-compatible, with the parallel sub-agent fleet the only behavior change and the only one behind a default-off flag.

5 Evaluation

The evaluation design*—the grading oracle, the task specifications, and the RQ structure—was fixed and committed in-repo (*`cgb_eval/`*)* before any model run*, so grading cannot be tuned to results. The Capability-Gap Benchmark (CGB) was then executed on systemu release* ***v0.9.41: 179 trials*** *across* ***five models / five vendors*** *and up to three conditions (June 2026, OpenRouter-hosted models; the matrix is 180 cells, one of which the* opus *spot-check dropped at its token cap). The tables below report the* realized *run; every number is recomputed by the pre-registered external oracle over durable artifacts (aggregation in* `cgb_eval/paper_numbers.py`*). Three procedural points are disclosed in full (§5.7):* (i) *the cross-trial MCP-registry leak found in an earlier run is* fixed at source *in v0.9.41—we verified it, every* push *MCP cell now correctly fails to reach the server, so unlike that earlier run* no post-hoc correction is applied to these numbers*;* (ii) *the pull-failure taxonomy was re-derived from the durable per-trial governor ledgers with the committed extractor (a resume crossed an extractor update, so we recompute uniformly from disk—also what makes the taxonomy reproducible from the published artifact); and* (iii) *the unattended batch auto-approved v0.9.41's new per-operation file-read confirmation prompt, matching the free file reads of the prior run.*

We evaluate the pattern language through its reference implementation on a controlled benchmark designed around the spine: *pull-decision quality* first, then efficacy, governance cost, the bounded safety properties that actually hold, and breadth across the six capability families. The single biggest credibility lever is that the system does *not* grade itself; success is decided by a pre-registered, independent oracle (§5.2).

5.1 Research questions

RQ1 — Pull-decision quality (primary). Does the agent correctly detect that it is blocked and request the *right* capability? We operationalize this as the *recognition rate*—a correct-kind harness request fired per pull run, read from `harness_kind` in the per-iteration decision audit—cross-validated against downstream recovery (RQ2). This is the central question (C3). The v0.9.41 build also reconciles each request to a terminal outcome in the governor ledger (§4.5), so we report the finer pull-failure taxonomy alongside recognition—grant disposition (used / unused / not-applicable), premature requests, wasted requests, and over-delegation denied at the per-run cap (§5.6). Earlier builds emitted these terminal events only across the suspend→resume boundary unreliably; that gap was localized by the benchmark and closed at source (§5.3).

RQ2 — Efficacy. Does runtime pull-provisioning recover tasks that a frozen-harness *push* baseline fails on due to capability gaps? Measured as capability-gap recovery rate, judged by the independent oracle.

RQ3 — Governance cost. What is the overhead of the proportionate arbiter versus the push baseline (tokens, LLM calls,

dollars), and how does that overhead split between deterministic and LLM-judged decisions? The split is read directly from the structured `decided_by` ledger field.

RQ4 — Bounded safety. Do the safety properties that we claim actually hold under fault injection? Specifically: HIGH-risk requests always escalate *regardless of any auto-grant flags*; the judge can only downgrade; and a Governor or judge failure can only deny or escalate, never grant. *(Stated narrowly: there is no cross-run-leakage claim—leases are attribution plus revocation, not containment; see P4.)*

RQ5 — Breadth across families. Does one governance path hold across all six capability families (TOOL, SKILL, ACCESS, COMPUTE, SUBAGENT, MCP)? We report per-family pull-decision quality and efficacy, and we are explicit that the *evidentiary weight differs by family*: TOOL (synthesis), MCP (attachment), and COMPUTE (budget) are materialized and measured end-to-end; SKILL is measured indirectly; SUBAGENT is real but opt-in; and ACCESS is arbitrated and logged, not materialized on the default backend (§5.3). TOOL and MCP are the two halves of the acquisition layer—synthesizing a capability versus attaching a vetted external one—under the *same* arbiter. The CGB also ships an ablation condition (`pull_min_governance`: judge off, adherence free) so the judge-on/off contribution can be read against the governed *pull* condition; we designate the judge-on/off contrast as the primary ablation and treat finer knobs (adherence dial, per-family auto-grant flags) as exploratory given the trial budget (§5.7).

5.2 The Capability-Gap Benchmark

The CGB is a curated suite of **23 tasks across the six capability families** (TOOL ×5, SKILL ×4, ACCESS ×4, COMPUTE ×3, SUBAGENT ×4, MCP ×3; `cgb_eval/tasks/`), where each task is solvable *only if* the agent pulls a capability it lacks. The TOOL tasks deliberately span *two* synthesis operations—four cryptographic-hash/checksum tasks and one compression task—so the synthesis result does not rest on a single operation a model might idiosyncratically (mis)handle. The sixth family, MCP, exercises the *attachment* half of the acquisition layer—governed connection to an external Model Context Protocol server—alongside the *synthesis* half the TOOL family exercises (forging new code); both flow through the same Governor.

Real gaps, not synthetic ones. A central design commitment is that the gap be *genuine*. An early version withheld a file-writer to force a tool request—but systemu ships file-writing, so the "gap" was an artifact of removing a capability the runtime already has, which makes the push-versus-pull efficacy comparison near-tautological (and, worse, the runtime's built-in writer *masked* the gap so the agent never recognized it). Instead each TOOL task withholds something the runtime *genuinely lacks* and an LLM *cannot fabricate*: a cryptographic hash/checksum the agent must forge a tool to compute, with the oracle recomputing the expected digest from the input so the answer cannot be guessed. The other families follow the same principle—a genuinely-absent iteration budget (COMPUTE), a non-obvious externally specified procedure (SKILL), a governed resource with no local reach (ACCESS), a breadth no single agent can cover within budget (SUBAGENT), and—for MCP—a datum that exists *only* behind an external service (a hermetic stdio MCP server returning an unguessable lookup code), so the agent must attach the integration and call its tool; the oracle recomputes the expected code from the same pure logic the server runs, so it cannot be fabricated. Each task declares its withheld capability and external success criterion as data on a frozen `CGBTask` record (`cgb_eval/task_spec.py`); the same Scroll/Activity is used across conditions so the comparison is a controlled A/B. The *push* condition receives the frozen, gap-bearing harness; the *pull* condition can request the missing piece.

Modeled operator approval (how RQ2 is measured under the safety rule). The most load-bearing pull—forging new tool code—is HIGH risk, so the Governor *escalates* it and the run suspends awaiting a human (the very property RQ4 checks). To measure recovery (RQ2) in an unattended batch, the harness models the operator's *approve*: it materializes the grant, stamps the resume snapshot, and drives the runtime's existing resume-after-grant path (`cgb_eval/operator.py`). This models a *yes* only; it never auto-grants inside the Governor and never suppresses the escalation, which remains recorded in the ledger for RQ4. We report RQ2 (recovery *given* approval) and RQ4 (the escalation fired) separately, and disclose the modeling.

Pre-registered, independent oracle (non-negotiable). The system's own goal-verifier is known to *over-accept* (a documented v0.9.7 limitation), so the runtime must *not* grade itself—that would be circular and is the most common way a self-provisioning result is rendered uninterpretable. We make the grader independent in two senses. *(i) Construction:* each task ships an *external ground-truth check*—artifact assertions over the durable workspace (file existence/content/line-count, JSON field equality, and their conjunction; `cgb_eval/oracle.py`), *not* the runtime's goal-verifier. The oracle reads only workspace artifacts a third party could inspect; it never reads the runtime's self-assessment. *(ii) Pre-registration:* the oracle's decision rule and each task's success criterion are committed in-repo before any model run, so pass thresholds cannot be tuned to results post-hoc. The grading code makes no network or model calls. Family-specific oracles match the materialization gradient (§5.3): TOOL/SKILL/COMPUTE/MCP tasks are graded on the produced artifact (for MCP, the file must contain the unguessable codes only the attached server can return); SUBAGENT tasks grade the collated (partial-credit) deliverable from *real parallel* execution; and ACCESS tasks grade *arbitration* (a completed task *or* a correct ACCESS request in the harness ledger), not materialization, in line with the honest family characterization (§4.4).

5.3 The materialization gradient (breadth caveat)

The "six families under one governance path" claim (C4) is honest only when paired with *how much of each family is actually materialized and measured*. The families sit on a gradient, and efficacy measurement is strongest at the top. *Three* families are end-to-end—the grant materializes a capability the run then uses, and the oracle grades the resulting artifact—and together they cover *both* halves of the acquisition layer (synthesis and attachment):

- **TOOL** (synthesis) — end-to-end: a granted request forges, dry-runs, and deploys a callable tool, and the oracle grades the artifact it produces.
- **MCP** (attachment) — end-to-end: a granted request connects a vetted external MCP server (a new server escalates HIGH and is materialized only on approval), discovers and definition-hash-pins its tools, and the agent calls them *this run*; the oracle grades an artifact that can contain only values the attached server returned.
- **COMPUTE** (budget) — end-to-end: a granted request raises the loop's iteration/think budget (clamped to the policy ceiling), which the loop actually consumes; graded on the finished artifact alone, with no request-as-success escape.
- **SKILL** — *indirect*: a grant authors and persists a skill, but the agent must subsequently load it to benefit, so the effect is measured one step removed from the grant.
- **SUBAGENT** — *real but opt-in*: a grant spawns the real parallel child fleet, but only behind the default-off flag `SYSTEMU_DELEGATE_USE_PARALLEL`, which we set on for the CGB run; with the flag off the family falls back to a one-shot stub.
- **ACCESS** — *arbitrated and logged, not materialized*: a grant records an *advisory* lease only; on the default local backend it opens no filesystem/network channel and enforces no isolation (that exists only on the Docker backend), and the v0.9.34 grant message now says so honestly rather than implying a channel was opened. We therefore measure ACCESS at the request→verdict layer, not at materialization.

The gradient bounds what each family can *materialize*. A separate, empirical boundary—which the benchmark surfaced and we report as a first-class RQ1 finding—bounds what the agent will *request*: **pull-decision quality is gated by gap perceivability**, and the benchmark lets us show this *longitudinally*. The agent reliably self-provisions when the gap manifests as a concrete failure it actually hits—a missing tool (TOOL: it tries to compute the hash, finds no tool, and asks), an unreachable resource (ACCESS), a breadth that overwhelms one agent (SUBAGENT), or a needed datum behind an unattached integration (MCP). The COMPUTE budget gap is the instructive boundary case, and we are deliberate about what we *measure* here versus what we infer. The measured claim is *within-version*: on the runtime evaluated here, the per-iteration payload surfaces the remaining budget (`iterations_remaining` plus an escalating low-budget notice), and the agent self-provisions compute unprompted—at a model- and trial-dependent rate the per-model RQ1 breakdown reports, not as a guarantee. The cross-version history is *motivation*, not a controlled experiment, and we flag it as such: an earlier runtime exposed no budget at all and agents did not request compute even when the goal told them to, which is what led us to localize the gap as a runtime *perceivability* defect—in the payload, not the agent's reasoning—and fix it. We deliberately do not rest a measured claim on that earlier-versus-current difference, since it changes more than one variable; the load-bearing evidence is the within-version observation that, once the budget is perceivable, the request appears with no goal hint. A second, independent defect sat behind the first: the granted budget defaulted to *zero* unless the agent named an amount the prompt never documented, so an unprompted request was a no-op; both the prompt gap and the zero-default were fixed (v0.9.34.1), after which a granted request actually extends the budget and completes the task. The remaining boundary is SKILL: a procedure the agent tends to attempt *inline* rather than provisioning. The pull-decision-quality measurement itself (RQ1) applies across all six families—it is read from the harness ledger and the per-iteration decision audit regardless of what a grant materializes—so we can quantify exactly where the agent does and does not recognize blockage. We surface this gradient, and the perceivability boundary, wherever the six-families count appears, so the headline number never outruns the evidence.

Diagnostic value, and why it is not teaching to the test. Because the benchmark exercises the production `ShadowRuntime` rather than a mock, its development surfaced *eight* concrete runtime defects that a static or goal-verifier-graded harness would have missed—each reproduced *outside* the eval through the normal daemon execution path, reported, and fixed before the scored run. We are explicit that this could be mistaken for tuning a system until it passes one's own test, and it is not, for two reasons we make checkable. *First*, every fix *restores the implementation to the design already specified* in §3–§4: the documented per-run request cap was dead code and was wired up; the MCP attachment path connected a server but failed to persist its transport spec, leaving its own tools uncallable. None changed the benchmark's tasks, injected gaps, or grader. *Second, and decisively*, the oracle's decision rule and each task's success criterion were committed in-repo *before any model run* (§5.2) and read only durable artifacts, so the bar a run must clear is fixed independently of any change to the system under test—"fixed before the run" describes the runtime, never the grading. Five defects were latent in the shipped v0.9.31 release (an undispatchable built-in toolset; the dead request cap; an unbounded sub-agent cascade; the COMPUTE perceivability defect above; a misleading ACCESS grant message); three more surfaced when reconciling to the MCP-bearing v0.9.34 build (an advertised-but-unhandled tool-search affordance, the zero-default COMPUTE grant, and the MCP transport-persistence defect found immediately by the new MCP family). A *ninth* surfaced in post-hoc analysis of an *earlier* scored run (on v0.9.34.4) and we treated it differently, as a *correction* rather than a pre-run fix: an in-memory MCP-registry leak let a `lookup` server attached during one trial remain callable in four later same-process *push* cells, spuriously passing them. We caught it because the easier three-key task failed in *push* while the harder four/five-key tasks "passed," traced it to a process-global registry not scoped per run, and *fixed it at source* in v0.9.36 (a once-per-run terminal teardown of the run's MCP tools); on the v0.9.41 run reported here we re-verified the fix holds—every *push* MCP cell correctly fails to reach the server—so *no correction is applied to these numbers*. A *tenth* item was the last to close, and the benchmark localized it precisely: on a *resumed* run (every HIGH-risk forge/attach/delegate escalates, suspends for approval, then resumes under a *fresh* execution id), an earlier terminal finalize reconciled only the resumed id while the request and lease lived under the original, suspended id—so the request-outcome taxonomy was blind for exactly the suspend-bearing families, which is why an earlier draft reported recognition alone. This was fixed at source across v0.9.38–v0.9.41: reconciliation now fires at *any* genuine terminal (including the resumed id), the per-run sub-agent cap was bounded and its over-delegation

denials now write a first-class ledger row, and `attempts_before` and `confidence` are populated. That closure is exactly why this run reports the full request-outcome taxonomy (Table 5) alongside recognition, re-derived from the durable per-trial ledgers. We report all of this not as a result of the pattern language but as evidence that a benchmark which drives the real loop and grades externally has *diagnostic* value: the same instrumentation that measures pull-decision quality also localizes where the implementation diverges from the design.

5.4 Conditions, models, and trials

Conditions (three). (a) *push*: the frozen-harness baseline (legacy intent engine, reverse-harness off); (b) *pull*: reverse-harness on with full governance (LLM judge on, guided adherence); (c) *pull_min_governance*: an ablation that minimizes governance within the supported configuration (judge off, adherence free), isolating the judge's contribution to safety and cost. In the pull conditions, per-family auto-grant is enabled so unattended runs never block on an operator card, and `SYSTEMU_DELEGATE_USE_PARALLEL` is on so a granted SUBAGENT request spawns the real parallel fleet; the HIGH-risk band still escalates regardless of these flags (RQ4). The arbiter's verdict distribution is read from the ledger, so escalation behaviour is measured even when auto-grant is on. To hold the base prompt as constant as possible across conditions, the *pull* prompt adds only the REQUEST_HARNESS verbs over the *push* prompt; we note this control because otherwise an efficacy gain could be a prompt artifact rather than the provisioning inversion.

Models. We span **five models across five vendors**: `google/gemini-3-flash-preview` (Google), `deepseek/deepseek-v4-pro` (DeepSeek), `openai/gpt-5.4` (OpenAI), `anthropic/claude-opus-4.8` (Anthropic), and `z-ai/glm-5.2` (Z-AI). Each is mapped to all three internal tier slots so a trial runs end-to-end on a single model. *gemini* and *deepseek* are run at full coverage; the other three are spot-checked on one task per family (§below). A sixth candidate, `nvidia/nemotron-3-ultra-550b-a55b:free`, was excluded and we disclose it rather than carry a non-functional cell: it returned malformed responses that crashed the runtime's response parser on every trial. (One operational note: *deepseek* exhibits a persistent response-format-repair loop that starves it under parallel execution, so it is run *solo*; this affects wall-clock, not its results.) Spanning a fast low-cost model, a capable low-cost model, and three frontier models across five vendors retires the free-tier "model-capability ceiling" confound flagged in the v0.9.7 assessment, so a failure to pull is attributable to the design or to the model's own gap-recognition rather than to a model too weak to engage, and lets us separate "the design works" from a single-model idiosyncrasy.

Agent instructions (disclosed). The executor's standing instructions include a *generic* clause directing the agent to acquire capabilities it lacks rather than fabricate an unverifiable result. We disclose it because the quality of the agent's instructions is itself a determinant of pull-decision quality (RQ1): an executor told not to fabricate *tool outputs* but not its own *computed values* will let an overconfident model emit, say, a fabricated hash instead of self-provisioning. The clause is part of the *shipped* agent (not an eval-time injection), names no task, no operation type, and no specific action, and is held constant across *push*, *pull*, and *pull_min_governance*—so it cannot account for any push-versus-pull difference. It is an instruction-quality property of a well-designed agent, not task coaching.

Trials and budget. The matrix is 23 tasks × up to 3 conditions × 5 models. Because per-token prices vary by nearly two orders of magnitude across vendors, spend is bounded by a *per-model* token cap rather than a fixed trial count, and each model runs as its own capped sweep into one resume-safe results file. We run *breadth-first*—one trial per cell—so the budget buys task/condition coverage rather than depth: *two* low-cost models (`gemini-3-flash` and `deepseek-v4-pro`) cover all 23 tasks across all three conditions (including the judge-on/off ablation), and the three frontier/diversity models contribute a one-task-per-family check on the push-versus-pull core (covering both a hash and the non-hash compression task). Once a model crosses its cap the runner stops scheduling further trials for it, so total spend is bounded regardless of nominal cell count; the runner is resume-safe and seeds each model's spent-token total from prior results, so an interrupted run continues without double-spending. We report the *realized* per-cell trial and per-model condition coverage with the populated tables. Results are reported as proportions with bootstrap confidence intervals, with exact (binomial) McNemar tests on paired push-versus-pull outcomes per (task × model); per-family numbers are a secondary, exploratory breakdown (§5.7). We are explicit that this is a *pilot-scale* run: breadth-first single trials support a characterization with confidence intervals, not fine-grained comparative claims (§5.7).

5.5 Metrics

- **Pull-decision quality (primary):** the recognition rate (correct-kind harness request fired per pull run, from the decision audit's `harness_kind`), cross-validated against recovery; and the request-outcome taxonomy reconciled in the governor ledger—grant disposition (used / unused / not-applicable-by-kind), premature requests, wasted requests, and over-delegation denied at the per-run cap. Both are re-derived from the durable per-trial ledgers by the committed extractor (§5.6).
- **Efficacy:** goal success (external oracle) and capability-gap recovery rate.
- **Cost:** tokens, LLM calls, and dollars; governance overhead in isolation; and the measured deterministic-versus-LLM split from the `decided_by` field.
- **Bounded safety:** via fault injection—inject judge timeouts and malformed output and confirm ESCALATE-not-grant; confirm HIGH-risk is never auto-granted *regardless of auto-grant flags*; confirm a Governor failure can only deny or escalate. (No cross-run-leakage metric; see P4.)
- **Accountability:** ledger completeness—every run's self-assembled harness (requests, verdicts, lease-mint/revoke events) is reconstructable from the ledger alone, validating P6.

5.6 Results

Numbers below are from the realized run: 179 trials over five models (two at full 23-task coverage—`gemini-3-flash` *and* `deepseek-v4-pro`*—and three spot-checked on one task per family), graded by the pre-registered external oracle. Proportions carry percentile-bootstrap 95%*

Table 4: RQ1 (primary): pull-decision quality (recognition). *Pull runs* = *pull*+`pull_min_governance` trials; *Requested* = runs firing ≥1 harness request; *Correct-kind* = runs requesting the family's own capability type (from `harness_kind` in the decision audit); *Recognition* = correct-kind/pull runs; *Recovery* = paired *pull*-over-*push* recovery (recovered/push-failures). The finer request-outcome taxonomy is in Table 5. Overall is the primary; per-family rows are exploratory (small *N*).

Family	Pull runs	Requested	Correct-kind	Recognition	Recovery
TOOL	26	17	17	65.4%	9/16
SKILL	19	4	0	0.0%	1/9
ACCESS	19	19	19	100%	11/11
COMPUTE	15	3	2	13.3%	0/9
SUBAGENT	19	16	16	84.2%	8/9
MCP	14	14	14	100%	8/8
Overall	112	73	68	60.7%	37/62

Table 5: RQ1 (second-order): request-outcome taxonomy reconciled from the governor ledger (re-derived from the durable per-trial ledgers by the committed extractor). *Premature* = requests fired with no prior tool attempt; *Cap-denied* = over-delegation denied at the per-run cap; *Grants* are split used / unused / not-applicable-by-kind (N/A = advisory ACCESS leases and SUBAGENT fleet spawns, where per-grant use is unobservable); *Used-grant* rate is over used+unused grants only (– where all grants are N/A). Per-family rows are exploratory (small *N*).

Family	Requests	Premature	Cap-denied	Grants (u/x/n/a)	Used-grant
TOOL	29	100%	0%	18 / 10 / 1	0.64
SKILL	4	100%	0%	1 / 3 / 0	0.25
ACCESS	25	0%	0%	0 / 0 / 25	—
COMPUTE	5	60%	0%	0 / 3 / 2	0.00
SUBAGENT	104	0%	17%	0 / 0 / 86	—
MCP	14	0%	0%	14 / 0 / 0	1.00
Overall	181	20%	10%	33 / 16 / 114	0.67

CIs; paired push-versus-pull contrasts use an exact (binomial) McNemar test pooled across all (task × model × trial) pairs, since the breadth-first single trial per cell makes the per-cell test degenerate (we disclose the pooling). Per-family rows are an exploratory breakdown at small N (§5.7).

RQ1 — Pull-decision quality (primary). Table 4 gives the primary result: *does the agent recognize it is blocked and request the right capability?* The headline measure is the *recognition rate* (a correct-kind harness request fired, per pull run), cross-validated against downstream *recovery*; the finer pull-failure taxonomy (Table 5) is reported next. Recognition is the load-bearing RQ1 finding, and it is itself a clean perceivability gradient: ACCESS, MCP (100% each) and SUBAGENT (84.2%) at the top, TOOL (65.4%) in the middle, then COMPUTE (13.3%) and SKILL (0%) at the bottom—the agent never frames the SKILL gap as a *skill* request, instead attempting the procedure inline (when it requests at all, 4/19 runs, it mis-frames it as another kind). Recognition broadly predicts recovery—ACCESS (11/11), MCP (8/8) and SUBAGENT (8/9) lead both, and TOOL (9/16) follows—with two instructive departures at the floor: SKILL recovers a single baseline failure (1/9) despite *never* being requested as a skill (it is occasionally solved inline), while COMPUTE is never recovered (0/9) even though it is sometimes recognized—an unmissable concrete gap (MCP, ACCESS) is both recognized and recovered, whereas a soft budget ceiling is neither reliably. The through line is the mechanism the paper claims: the reverse-harness works *when invoked*; the bottleneck is invocation, i.e. perceiving the gap. Per-family rows are an exploratory breakdown (small *N*; §5.7).

Request-outcome taxonomy (RQ1, second-order). Table 5 reconciles every request to its terminal disposition in the governor ledger—the finer lens that earlier builds could not capture across the suspend→resume boundary (§5.3), now closed at source and re-derived from the durable per-trial ledgers. The accounting is clean: the 181 requests resolve to 163 grants (90%) and 18 over-delegation denials at the per-run cap (10%), with *no* wasted requests (a deny where a viable fallback existed) and *no* unused grants left on the table. Three findings stand out. *(i) Over-delegation is real and bounded.* The SUBAGENT family alone fires 104 of the 181 requests—a mean of 5.5 per run over its 19 runs, the agent eagerly fanning out—and the per-run cap denies 18 of them (17%), which is the cascade bound (P-series safety property) doing its job under load; every other family's cap-denial rate is 0. *(ii) Grant "use" is observable only where the family materializes something callable.* Of the 163 grants, 33 are confirmed used and 16 unused, but 114 are *not-applicable by kind*—dominated by SUBAGENT fleet spawns (86) and ACCESS advisory leases (25), with the remaining 3 in TOOL/COMPUTE, where there is no single downstream call to mark as "used"; the used-grant rate is therefore reported only for the materialize-and-call families, where it is 0.64 (TOOL), 0.25 (SKILL), and 1.00 (MCP). *(iii) Premature requests track immediacy, not waste.* The runtime's classifier marks 20% of requests premature (fired with no prior tool attempt), concentrated entirely in TOOL (100%), SKILL (100%), and COMPUTE (60%)—the families with no inline fallback to try first, where asking on first recognizing the gap is appropriate—so we report the rate but read it as request immediacy rather than a quality defect. Across all 181 requests the arbiter decided *every* one on the deterministic risk-band path (0 reached the LLM judge), which is the RQ3 cost result previewed here. Overall, the agent fires a correct-kind request on 60.7% of pull runs and those requests convert to a 59.7% recovery of baseline failures (Table 6).

RQ2 — Efficacy. Table 6 reports goal success (external oracle) per condition and the capability-gap recovery rate of *pull* over the *push* baseline, with bootstrap CIs and exact-McNemar p on paired outcomes. Because end-to-end materialization is strongest for TOOL/MCP/COMPUTE (§5.3), the efficacy reading is most direct for those families. The headline is a large, consistent inversion: the frozen-harness baseline succeeds on 6.0% of gap-bearing tasks, runtime pull-provisioning on 60.6%. On the 66 paired (task × model) outcomes, *pull* wins where *push* fails 37 times and loses where *push* wins only once (exact McNemar $p \approx 2.8 \times 10^{-10}$), recovering 59.7% of the baseline's failures; the single regression is in SKILL,

Table 6: RQ2: goal success per condition and gap-recovery (*pull* over *push*). Success is the external oracle's verdict; CIs are percentile bootstrap; n is the per-condition trial count; gap recovery is recovered/push-failures on paired cells; McNemar is exact, pooled across paired (task × model × trial) outcomes.

Condition	Success (95% CI)	Gap recovery	McNemar p
push	6.0% (1.5–11.9), n=67	—	—
pull	60.6% (48.5–72.7), n=66	59.7% (37/62)	2.8×10^{-10}
pull_min_governance	58.7% (43.5–71.7), n=46	56.8% (25/44)	6.0×10^{-8}

Table 7: RQ3: governance cost per condition (token mean with bootstrap CI; mean LLM calls) and the deterministic/LLM arbitration split from `decided_by` (pooled across governed requests). Dollar cost is bounded by per-model token caps at June-2026 OpenRouter pricing (text).

Condition	Tokens (95% CI)	LLM calls	det. / llm
push	62.0k (50.4–74.7k)	10.9	—
pull	66.9k (55.8–79.2k)	12.7	181 / 0
pull_min_governance	77.9k (57.7–102.8k)	13.6	(judge off)

the family the agent tends to attempt inline. The effect holds in every one of the five models (per-model *push*→*pull*: gemini 2→13, deepseek 0→15, gpt 0→3, opus 1→4, glm 1→5 of their scored cells), so it is not a single-model artifact. The judge-off ablation (`pull_min_governance`) is statistically indistinguishable from full *pull* on efficacy (58.7% vs 60.6%, overlapping CIs), locating the judge's value in safety and accountability rather than recovery.

RQ3 — Governance cost. Table 7 reports tokens and LLM calls per condition, and the deterministic-versus-LLM decision split read directly from the ledger's `decided_by` field. Governance overhead is modest: *pull* adds ≈ 8% tokens and ≈ 17% LLM calls over *push*, which buys the 6.0%→60.6% recovery. Two findings about *where* the cost is not: first, of the 181 governed requests the ledger recorded, **all** 181 **were resolved by the deterministic risk-band table and** 0 **by the LLM judge**—in this suite the proportionate arbiter dispatched every decision deterministically (the judge is reserved for the AMBIGUOUS band, which the auto-grant policy did not route here), so the expensive judged path added no measured token cost. Second, the judge-off ablation costs *more*, not less (77.9k vs 66.9k tokens): with the judge and guided adherence off, runs flail longer, so the governance machinery is not the cost driver. Dollar cost is bounded by per-model token caps; at June-2026 OpenRouter prices the full 179-trial run cost ≈ $19, of which ≈ $10 is the single *opus* frontier spot-check (at $7.6/M tokens); the two low-cost full-coverage models, which carry the bulk of the cells at ≤ $0.83/M tokens, together cost ≈ $6.

RQ4 — Bounded safety (fault injection). Table 8 records the outcome of each injected fault. The property under test is that each fault can only *deny* or *escalate*, never *grant*: a judge timeout or malformed/boundary judge output must yield ESCALATE; a HIGH-risk request must escalate even with all auto-grant flags on; and a Governor failure mid-materialization must leave the ledger consistent. The safety argument reduces to the correctness of the deterministic band table (a hand-written classifier); a band misclassification is the residual un-judged risk, which the "HIGH never auto-granted" row directly probes.

Table 8: RQ4: bounded-safety properties under fault injection. Expected outcome for every row is DENY/ESCALATE (never GRANT) or a consistent ledger. All six properties hold (6/6). The suite is pure verification against the deterministic arbiter with a monkeypatched judge (no API calls; `cgb_eval/safety_export.py`).

Injected fault	Expected	Observed
Judge exception / timeout	escalate	escalate
Malformed judge output	escalate	escalate
Low-confidence judge (0.6 boundary)	escalate/deny	escalate
HIGH-risk, all flags on	escalate	escalate
Judge verdict can only downgrade	no upgrade	holds
Governor failure mid-materialize	consistent ledger	consistent

Table 9: RQ5: per-family breadth, paired with the materialization gradient (§5.3). Evidence is strongest for TOOL/MCP/COMPUTE. Efficacy is the *pull*-condition success rate (external oracle). The ordering tracks gap perceivability (§5.3); per-family N is small, so these are an exploratory breakdown (§5.7).

Family	Materialization	Efficacy (95% CI)
TOOL	end-to-end (synthesis)	56.2% (31–81), 9/16
MCP	end-to-end (attachment)	100% (8/8)
COMPUTE	end-to-end (budget)	0% (0/9)
SKILL	indirect (persist-then-load)	18.2% (0–45), 2/11
SUBAGENT	real, opt-in (flag-gated)	90.9% (73–100), 10/11
ACCESS	arbitrated + logged	100% (11/11)

RQ5 — Breadth across families. Table 9 summarizes per-family efficacy alongside the materialization gradient, so the breadth claim is read against its evidentiary weight. The result is a clean monotone gradient that tracks gap *perceivability*: MCP 100% (8/8) and ACCESS 100% (11/11)—concrete, unmissable gaps—then SUBAGENT 90.9% (10/11) and TOOL 56.2% (9/16), then SKILL 18.2% (2/11) and COMPUTE 0% (0/9) at the bottom. The two extremes are the instructive cases. MCP (attachment) and TOOL (synthesis)—the two halves of the acquisition layer—both recover through the *same* arbiter, which is the core breadth claim (C4). COMPUTE is the boundary: an invisible budget ceiling is recovered 0% even though the per-iteration payload now surfaces the remaining budget, because the agent must *notice* a soft limit it never visibly hits—the perceivability bound made concrete. The primary ablation (judge on via *pull* versus judge off via `pull_min_governance`) is reported from the same matrix; finer ablation knobs are exploratory (§5.7).

5.7 Threats to validity

Statistical power (stated plainly). This is a *pilot-to-modest* scale and we do not hide it. With 23 tasks (3–5 per family), a breadth-first single trial per cell, and per-model token caps that leave the diversity models covering only the push-versus-pull core, per-cell N is small. Per-family recognition / request-outcome rates (RQ1) over 3–4 tasks therefore carry wide confidence intervals that will not support fine-grained *comparative* claims. We mitigate by (i) treating the *overall* pull-decision quality as the primary result and per-family rows as an exploratory breakdown, both explicitly labelled in Table 4; (ii) reporting effect sizes with bootstrap CIs rather than point estimates alone; and (iii) using exact (not asymptotic) McNemar tests, which remain valid at small paired counts. The honest reading of RQ1 is a *characterization with confidence intervals*, not a claim of a precise number; a higher-powered run is future work.

Construct. The chief construct threat is oracle circularity. We address it with a pre-registered external checker per task (§5.2): the oracle's decision rule and pass criteria are committed in-repo before any run (so they cannot be tuned to results), it reads only durable workspace artifacts, and it never reuses the runtime's goal-verifier or any runtime self-assessment. The runtime never decides its own success.

Internal. Model and run-to-run variance could confound outcomes; we span five models across five vendors and report confidence intervals with paired tests (a breadth-first single trial per cell limits within-cell variance estimates, which we flag as a power limitation above). A second internal threat is a *prompt* confound: because the *pull* condition's prompt adds the REQUEST_HARNESS verbs, an efficacy gain could in principle be a prompt artifact. We control for this by holding the base prompt constant across conditions and adding only the pull verbs to *pull* (§5.4); a residual prompt effect cannot be fully excluded and we flag it as such. A third internal threat is *cross-trial state leakage*: because each model's sweep runs in one process, process-global state can bleed between trials. An *earlier* run hit exactly one instance—an MCP connection attached in a *pull* trial remained callable in four later same-process *push* cells—detected from the failure pattern (the easier task failed in *push* while harder ones "passed") and *fixed at source* in v0.9.36 (a once-per-run terminal teardown of the run's MCP tools). The v0.9.41 run reported here was re-verified clean: every *push* MCP cell fails to reach the server, and the other families show no analogous signature (TOOL/ACCESS/COMPUTE *push* are all 0, so no grant leaked), so *no correction is applied to these numbers.*

Instrumentation fidelity. The pull-failure taxonomy depends on the runtime emitting a terminal request-outcome reconciliation per request. Earlier builds emitted it unreliably across the suspend→resume boundary—a resumed run finalized a fresh execution id, missing the original's requests—which is why an earlier draft reported recognition alone. v0.9.38–v0.9.41 closed this: reconciliation now fires at *any* genuine terminal (including the resumed id), over-delegation denials write a first-class ledger row, and `attempts_before/confidence` are populated, so the full taxonomy (Table 5) is reported. Because the run was *resumed* across an update to the eval-side extractor, we recompute the taxonomy *uniformly* from the durable per-trial governor ledgers with the committed extractor rather than trust the heterogeneous run-time-captured values; the recomputation reproduces every run-time count exactly and only fills the previously-missing not-applicable-by-kind grants, and it makes the taxonomy reproducible from the published artifact. The residual limitation on RQ1 is statistical power (small per-family N, above), not instrumentation; recognition, the externally-graded efficacy (RQ2), and safety (RQ4) never depended on it.

External. Gap-injection is synthetic, so results may not transfer to naturally occurring gaps. We mitigate by spanning all six families. A separate external-validity limit is the *materialization gradient* (§5.3): end-to-end efficacy evidence is strongest for TOOL/MCP/COMPUTE, weaker for SKILL/SUBAGENT, and absent at the materialization layer for ACCESS; the breadth claim should be read against this gradient, not the bare count of six.

Honesty. We state the goal-verifier precision tradeoff explicitly (the system over-accepts, which is exactly why grading is external) and we restrict the safety claims to the properties the implementation actually holds (§5.1, RQ4), making no containment or cross-run-leakage claim. Every results cell reports the realized 179-trial run, recomputed by the pre-registered external oracle over durable artifacts; the build now instruments the full request-outcome taxonomy, which we report and re-derive reproducibly from the durable ledgers rather than estimate; the cross-trial MCP leak found in an earlier run is fixed at source and re-verified clean here, so no correction is applied to these numbers; and we disclose that the unattended batch auto-approved v0.9.41's per-operation file-read confirmation, matching the prior run's free file reads and applied identically across conditions, so it cannot account for any push-versus-pull difference. We report no number the run did not produce.

6 Discussion

6.1 Proportionate governance, not containment

The central design stance of this work is that self-provisioning under single ownership calls for *proportionate* governance rather than containment. A containment story—default-deny sandboxing, forced lease expiry, treating the agent as an adversary to be jailed—answers a question we are not asking. Under the single-owner, local-deputy threat model (§1.2) the agent holds the owner's permissions by design; there is no boundary it is escaping, and the friction of containment fights the automation the tool exists to deliver.

The honest and, we argue, more interesting finding is about *how little* governance self-provisioning actually needs. The reversible majority of requests—reusing an existing tool, taking a small budget increment, reading a whitelisted resource—can be auto-granted deterministically, at zero human cost. Friction is worth paying only for the genuinely dangerous-and-irreversible: forging new code, escalating access, exceeding ceilings. Proportionality is the whole design: it is what lets autonomy and oversight coexist without either swamping the other. Where friction stops being worth it is an empirical question the governance-cost measurement (RQ3) is designed to answer, by quantifying how much of the arbitration load the cheap deterministic band carries versus the expensive LLM-judged band.

6.2 The goal-verifier precision tradeoff

We are explicit about a limitation we did not engineer away: the system's own goal-verifier *over-accepts*. It is tuned to avoid false rejections (telling the owner a task failed when it succeeded), and the price of that choice is false acceptances. This is precisely why the evaluation grades success with an *external* oracle (§5.2) rather than trusting the runtime's self-assessment. We surface the tradeoff rather than hide it because it is load-bearing for interpreting any self-reported success number from a self-provisioning system: such numbers must be read as the system's own optimistic estimate, not as ground truth. The right response is not to claim the verifier is precise; it is to never let the verifier grade the experiment.

6.3 The threat-model boundary

Our guarantees are scoped to single ownership. We state plainly what changes if the boundary moves. Under multi-tenancy—several mutually distrustful principals sharing a runtime—the "agent acts with the owner's permissions" premise breaks, and the descoped concerns return as real requirements: access would need genuine enforcement (the optional Docker hardening becomes mandatory), self-grants would need per-tenant isolation, and the policy would need a notion of which principal a request is acting for. None of this is needed for the single-owner tool, and building it in would impose its cost on every user to serve a setting they are not in. We therefore treat multi-tenant policy as future work (§8) and keep the present contribution honestly scoped.

6.4 The residual injection risk

One narrow security residual remains even under single ownership: an agent that consumes *untrusted input* (a web page, an email, a file from an unknown source) and then takes an *irreversible or outbound* action could, in principle, be steered by prompt injection into provisioning or acting against the owner's interest. This is real but bounded, and it is orthogonal to the push/pull inversion—it exists for any agent that mixes untrusted input with consequential action. Risk-Tiered Arbitration (P3) already routes the most dangerous actions to operator escalation, which blunts the worst outcomes. A stricter posture—e.g., requiring confirmation when untrusted-input provenance meets an irreversible/outbound sink—is available as an *optional, off-by-default* control. We deliberately do not make it a default-deny sandbox: that would reintroduce the containment friction this design rejects, to defend against a residual that proportionate escalation already largely covers. Naming the residual openly, and offering an opt-in control for owners who want it, is the honest treatment.

6.5 Transferability of the patterns

Although validated by one implementation, the patterns are stated independently of it. Any supervised-agent runtime with an action loop can add an inverse verb (P2), route its self-requests through a risk-tiered arbiter with a downgrade-only judge (P3), keep that judge off the dispatcher's critical path (P5), persist self-built capabilities with provenance and one-click revocation (P4), and record the whole thing in an append-only ledger (P6). The reference implementation is evidence that the composition works end-to-end—a pulled capability is materialized and the loop consumes it across synthesis (TOOL), attachment (MCP), and budget (COMPUTE), with task-level efficacy following for TOOL and MCP, and COMPUTE the materialized-but-unrecovered boundary case (§5); the patterns themselves are the transferable contribution.

7 Related Work

Agent autonomy and tool use/creation. A large body of work equips LLM agents to use external tools—from self-supervised API calling [12] to mastering thousands of real-world APIs [10]—and a growing thread lets agents *create* tools at runtime rather than only invoke a fixed set: Voyager grows a reusable library of executable skills [15], and CREATOR has the model author its own tools via documentation and code [9]. Our inversion (P1) generalizes this beyond tools: tool creation is one family among six, and all six flow through a single governance path—and tool *synthesis* is paired with its dual, *attachment* of a vetted external MCP server, under that same path. Where prior tool-creation work largely assumes the agent should be able to make tools, we ask the orthogonal question of *how that self-provisioning should be governed*, and answer it proportionately rather than permissively or restrictively.

Agent metacognition and know-your-limits. The pull paradigm rests on the agent recognizing its own blockage and asking for the right thing—a metacognitive act. Work on calibration and self-knowledge—showing that large models are often well calibrated and can predict whether they know an answer [5], and can be taught to express that uncertainty in words [6]—is the relevant prior art. Our contribution here is empirical and specific: we instrument and measure pull-decision quality (C3) as a first-class property of a deployed system, rather than studying calibration in the abstract.

Human-in-the-loop approval. Routing consequential actions through human oversight is an established safety theme, from scalable-supervision framings of objectives too expensive to evaluate at every step [1] to recent analyses of approval gates that pause risky agent actions for a reviewer and the resulting fatigue and capacity limits [14]. Risk-Tiered Arbitration (P3) refines the binary "approve everything / approve nothing" into a three-band, default-deny scheme in which the human sees only the dangerous-and-irreversible minority—directly targeting the reviewer-capacity bottleneck—and the off-path judge (P5) keeps the approval machinery from stalling the dispatcher.

Reasoning-and-acting agent loops. The interleaving of reasoning and acting in agent loops [16], and its extension with verbal self-reflection over past attempts [13], is the substrate our inverse verb extends: `REQUEST_HARNESS` (P2) is a new action alongside reason and act, dual to the tool-call action. We situate self-provisioning as a first-class loop primitive rather than an out-of-band configuration step.

Capability security and least privilege. The principle of least privilege, complete mediation, and fail-safe defaults [11]; object-capability security as a discipline for authority and robust composition [8]; and time-bounded leases as a fault-tolerant grant primitive [4] are the security foundation we build on and *cite as prior art*, not as our claim. P3's default-deny mediation and P4's attribution-and-revocation reuse these ideas; the novelty is their application to an

agent provisioning capabilities *for itself* at runtime, plus the two P3 kernels (self-request asymmetry, downgrade-only judgment).

Governance and policy for autonomous systems. Work on visibility, audit, and accountability for AI agents—e.g., agent identifiers, monitoring, and activity logging as governance measures [2]—frames our accountability layer, and the Provenance Ledger (P6) applies the event-sourcing discipline of recording state changes as an append-only sequence of events [3] to the specific problem of reconstructing a self-assembled harness. The off-path split (P5) borrows the control-plane / data-plane separation popularized by software-defined networking [7] and applies it to an LLM-in-the-loop runtime.

Positioning. Across these areas, the distinct contribution of this paper is to treat *agent-initiated, runtime capability provisioning* as a first-class design concern with its own pattern language, to make its load-bearing assumption (pull-decision quality) measurable, and to govern it proportionately under an explicit single-owner threat model.

8 Conclusion and Future Work

We have argued that capability provisioning for supervised agents is mismatched to how agents actually get stuck: the deciding moment (design-time provisioning) precedes the knowing moment (the agent discovering, mid-task, what it lacks). We resolve the mismatch by inverting push into pull—the executing agent requests the capabilities it lacks, and an always-on authority materializes them under *proportionate* governance, frictionless for the reversible majority and a deliberate light touch for the dangerous-and-irreversible. We abstracted the design as a language of six patterns (P1–P6), spanning inversion, authority and arbitration, and accountability, with well-known security mechanisms cited as prior art and only the novel application and a small number of kernels claimed. We characterized the load-bearing assumption—*pull-decision quality*, that the agent knows when it is blocked and asks for the right thing—and made it measurable. We characterized self-provisioning's breadth across six capability families honestly—spanning both synthesis (forging a tool) and attachment (connecting a vetted external MCP server)—by what the reference implementation, systemu v0.9.41, actually materializes. And we laid out a controlled, independently-graded benchmark to evaluate the design, scoped to an explicit single-owner threat model.

The patterns are the transferable core: any supervised-agent runtime can adopt an inverse verb, a risk-tiered arbiter with a downgrade-only off-path judge, attributed-and-revocable self-grants, and a provenance ledger. The reference implementation is evidence the composition works end-to-end—the acquisition layer materializes a pulled capability the loop then consumes—across TOOL, MCP, and COMPUTE, read against the materialization gradient (§5.3).

Future work. Several directions extend the design beyond its current boundary.

- **Learned/earned auto-grant (the trust frontier).** Today trust compounds only by simple use: a capability that keeps working keeps existing. A natural next step is to promote capabilities from escalate-by-default toward auto-grant based on accumulated telemetry—earned auto-grant for request kinds that have a clean track record—widening the frictionless band safely as evidence accrues.
- **Docker-backend ACCESS enforcement.** The honest characterization of the access family (§4.4) leaves enforcement as an optional Docker hardening. Making default-deny network and scoped-mount enforcement a first-class, opt-in posture on the Docker backend would turn "arbitrated + logged" into "arbitrated + enforced" for owners who want it, without imposing it on the single-owner local default.
- **Multi-tenant policy.** Lifting the single-owner assumption (§6.3) requires a policy that knows which principal a request acts for, per-tenant isolation of self-grants, and mandatory access enforcement. This is the path from a proportionate-governance tool to a multi-tenant platform.
- **The governed-autotelism arc.** Reverse-Harness governs capabilities the agent pulls in service of a goal the owner set. A further arc governs an agent that proposes and pursues *its own* sub-goals under the same proportionate authority—governed autotelism (the "Muse"). That is the subject of a companion paper; the governance machinery developed here (P3, P5, P6 especially) is the foundation it builds on.

Availability. The reference implementation, systemu, is open source at https://github.com/rameswaran-mohan/project-systemu; the artifact for this paper is the public repository at commit `2150844f` (release **v0.9.41**), which materializes the reverse-harness runtime (Governor, deterministic arbiter, downgrade-only judge, parallel sub-agent fleet, MCP connection manager, and decision-audit ledger) across both halves of the acquisition layer (synthesis and attachment). The evaluation in §5 was *run* on this same release, v0.9.41. The Capability-Gap Benchmark—its task suite across the six families, the three experimental conditions, the pre-registered independent oracle, the modeled-operator resume, and the aggregation driver (`cgb_eval/paper_numbers.py`) that recomputes every number in §5—is released as the `cgb_eval/` package in the same repository at that commit, *together with the realized 179-trial per-trial results* (`cgb_results/v0941_*.jsonl`) and the durable per-trial decision audits and governor ledgers from which recognition and the pull-failure taxonomy are re-derived, so every reported number is reproducible from the raw data rather than merely re-runnable. A citable preprint of this paper is archived at https://doi.org/10.5281/zenodo.20816384 (CC BY 4.0).

References

[1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. arXiv:1606.06565 [cs.AI] https://arxiv.org/abs/1606.06565

[2] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. 2024. Visibility into AI Agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*. 958–973. arXiv:2401.13138 [cs.CY] doi:10.1145/3630106.3658948

[3] Martin Fowler. 2005. Event Sourcing. Online article, martinfowler.com. https://martinfowler.com/eaaDev/EventSourcing.html

[4] Cary G. Gray and David R. Cheriton. 1989. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP '89)*. 202–210. doi:10.1145/74850.74870

[5] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. 2022. Language Models (Mostly) Know What They Know. arXiv:2207.05221 [cs.CL] https://arxiv.org/abs/2207.05221

[6] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching Models to Express Their Uncertainty in Words. arXiv:2205.14334 [cs.CL] https://arxiv.org/abs/2205.14334

[7] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. 2008. OpenFlow: Enabling Innovation in Campus Networks. *ACM SIGCOMM Computer Communication Review* 38, 2 (2008), 69–74. doi:10.1145/1355734.1355746

[8] Mark Samuel Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* Ph. D. Dissertation. Johns Hopkins University. https://jscholarship.library.jhu.edu/handle/1774.2/873

[9] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. arXiv:2305.14318 [cs.CL] https://arxiv.org/abs/2305.14318

[10] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789 [cs.AI] https://arxiv.org/abs/2307.16789

[11] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939

[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL] https://arxiv.org/abs/2302.04761

[13] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 [cs.AI] https://arxiv.org/abs/2303.11366

[14] Emre Turan. 2026. Oversight Has a Capacity: Calibrating Agent Guards to a Subjective, Fatiguing Human. arXiv:2606.08919 [cs.AI] https://arxiv.org/abs/2606.08919

[15] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291 [cs.AI] https://arxiv.org/abs/2305.16291

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] https://arxiv.org/abs/2210.03629